CSC483/583 Programming Project: Building (a part of) Watson with a Three-Stage Retrieval Pipeline for Jeopardy!-Style Wikipedia Answer Selection

Cole Mobberly, Suhani Surana, Aunabil Chakma
University of Arizona

Abstract

We study a retrieval formulation of Jeopardy!-style question answering in which each clue must be mapped to the title of a Wikipedia page. We use a parsed Wikipedia collection of approximately 280,000 pages and index 152,042 non-redirect articles, and we evaluate on 100 Jeopardy clues whose gold answers are Wikipedia titles. We present a three-stage retrieval pipeline that uses increasingly expensive models only after the candidate set has been reduced. Stage 1 uses a weighted ranking of sparse and dense retrieval signals to produce an initial candidate list. Stage 2 reranks the top 100 candidates with a cross-encoder using natural-question reformulations of the clue. Stage 3 reranks only the top 10 candidates from the previous stage with Qwen3-Reranker-4B, a larger reranker model. This staged design improves the P@1 score from 0.31 after the initial weighted retrieval stage to 0.41 after cross-encoder reranking and to 0.47 after the final Qwen reranker. The main takeaway is that strong answer selection comes from combining broad early retrieval with progressively stronger semantic rerankers over much smaller candidate sets.

1 Introduction

Jeopardy!-style clue answering is a difficult retrieval problem because the clue is often indirect, category-dependent, and phrased differently from the text that appears in the target article. Rather than asking for a direct fact, many clues refer to an entity through paraphrase, role, quoted language, or historical description. This creates a lexical mismatch problem: the words in the clue are often not the words that appear most prominently in the correct Wikipedia page.

We treat the task as page retrieval over Wikipedia. Given a Jeopardy category and clue, the goal is to predict the Wikipedia page title that matches the gold answer. This avoids full generative question answering and turns the task into candidate generation and reranking over 152,042 indexed non-redirect articles. The main challenge is to retrieve the right page despite noisy Wikipedia markup, long article bodies, redirect pages, and clues whose phrasing differs sharply from natural encyclopedia prose.

Our core design principle is to use simpler and cheaper methods early, then apply larger semantic models later only to narrowed candidate sets. The resulting system is a three-stage retrieval pipeline. Stage 1 uses weighted hybrid retrieval, which combines BM25 ranking over cleaned article text with Dense Passage Retrieval (DPR) embeddings [Karpukhin et al., 2020] indexed in FAISS [Johnson et al., 2019]. Stage 2 reranks the top 100 candidates using a cross-encoder trained for passage ranking [Nogueira and Cho, 2019]. Stage 3 exports only the top 10 candidates from the combined second-stage ranking and reranks them with Qwen3-Reranker-4B, a larger reranker model [Qwen Team, 2025]. We also reformulate each clue into five natural-language questions so that the dense retriever and rerankers see inputs that better match question-versus-passage training distributions. Throughout the paper, Precision@1 (P@1) is the main evaluation metric because the system must return one final Wikipedia title as its answer.

This staged architecture yields consistent gains across the pipeline. The weighted hybrid retrieval stage reaches a P@1 score of 0.31, the combined second-stage ranking improves this to 0.41, and the final Qwen reranker reaches 0.47. These results support a simple conclusion: retrieval quality matters first, but stronger semantic reranking becomes effective once the correct page has been brought into a small candidate set.

2 Task and Dataset

Our task is to answer Jeopardy-style clues by retrieving the Wikipedia page whose title matches the gold answer. Each example contains a category, a clue, and one correct Wikipedia title. This makes the problem a page-ranking task rather than a fully generative question answering task: given the category and clue, we must rank candidate Wikipedia pages and place the correct title at the top.

We use a dataset of 100 Jeopardy clues whose answers appear as Wikipedia page titles, together with a parsed Wikipedia collection of approximately 280,000 pages. After separating redirects from ordinary articles, we index 152,042 non-redirect article pages. This setup creates two main challenges. First, the collection is large enough that retrieval quality matters immediately. Second, Jeopardy clues are often indirect and category-dependent, so the wording of the clue may differ substantially from the wording used in the correct Wikipedia article.

3 Core Implementation

3.1 Wikipedia Preprocessing

We index the collection with Whoosh, treating each Wikipedia page as a separate document rather than indexing one document per source file. This matches the requirement that the final answer must be a Wikipedia page title, so the indexing unit has to be the individual page. We store parsed pages in SQLite, keep redirect pages separate from article pages, and build the retrieval pipeline over page-level article records. In the final system, we index only the non-redirect article pages, which leaves 152,042 indexed documents out of the roughly 280,000 parsed pages.

We also prepared the text specifically for Wikipedia content. In the cleaning stage, we removed redirect lines, section headers, template markup, reference markup, file/image markup, and category lines, then normalized whitespace. Redirect pages were handled explicitly instead of being treated as normal article bodies, because they do not contain meaningful page content and should not be indexed as ordinary evidence-bearing articles in the final system.

For term preparation, we tried two paths. The first was a custom normalization pipeline built in our preprocessing code: lowercasing, whitespace tokenization, boundary-punctuation stripping, stop-word removal using spaCy's English stop-word list, removal of non-word characters, and splitting on underscores and hyphens. The second was to index the cleaned article text directly with Whoosh's default analyzer, which applies its own tokenization, lowercasing, stop-word filtering, and stemming. In our experiments, the default Whoosh analyzer was better, so the final system uses that default analyzer rather than the custom tokenized representation.

3.2 Indexing

We create multiple sparse and dense indexes so that different evidence sources can be tested inside one weighted retrieval framework. Table 1 summarizes the index variants we built.

Beyond these indexes, we also implemented two simple pattern-based bonuses: one checks whether a four-digit year from the clue also appears in the candidate article, and the other checks whether quoted text

Table 1: Indexes used in the retrieval experiments. Except the top mentioned index, all are processed by Whoosh's default analyzer.

Index	Description
Body only index	Body text indexed (proccessed with custom tokenization and normalization)
Joint title+body index	sparse index over article titles together with article body text
Redirect index	redirect-page titles stored as alternative page-name strings
First-sentence index	sparse index over only the first sentence of each article
First-two-sentences index	sparse index over only the first two lead sentences
First-paragraph index	sparse index over only the opening paragraph text
Category index	sparse index over article category labels linked to each page
DPR first-sentence FAISS	dense vector index over title plus first-sentence text
DPR first-two-sentences FAISS	dense vector index over title plus first two sentences
DPR first-paragraph FAISS	dense vector index over title plus opening paragraph
DPR entire-article FAISS	dense vector index over title plus full cleaned article text
Qwen-summary FAISS	dense vector index over Qwen-generated article summaries

from the clue appears in the candidate article. We also tested category-enriched dense variants that append article category labels to the dense text representation. Because Jeopardy clues are often fragments rather than natural questions, we also rewrote each clue into five natural-language questions with Qwen3-8B. We used these generated questions as dense-retrieval queries in stage 1 and as inputs to the later reranking stages. Appendix Figure 2 lists the text templates for inputs and prompts used for our system.

3.3 Stage 1: Weighted Hybrid Retrieval

In stage 1, we build an initial ranking over the full Wikipedia collection using the category text plus the clue text as the query. This stage combines one sparse retrieval score with three dense retrieval scores and is responsible for broad candidate generation before the more expensive rerankers are applied.

For the final selected run, the active stage-1 components are:

- BM25 over custom-processed article body text, weight 15.00
- DPR retrieval for the category+clue query, weight 1.00
- DPR retrieval averaged over five generated natural questions, weight 1.00
- DPR retrieval for the concatenation of those five generated natural questions, weight 1.00

Each DPR retrieval score is computed over four dense document indexes: title plus first sentence, title plus first two sentences, title plus first paragraph, and title plus full article text. For a given query, we keep the maximum score returned by these four DPR indexes for each candidate page. In the natural-question mode, we first run DPR separately for each of the five generated questions and then average those candidate scores. We retained title, redirect, lead-only, Qwen-summary, year-based matching, quoted-text matching, and category-enriched dense variants in the codebase, but all of them received zero weight in the final configuration.

This stage gives us a strong but still inexpensive global ranking. It searches the whole collection, surfaces the top candidates for each clue, and supplies the top 100 pages to the cross-encoder stage.

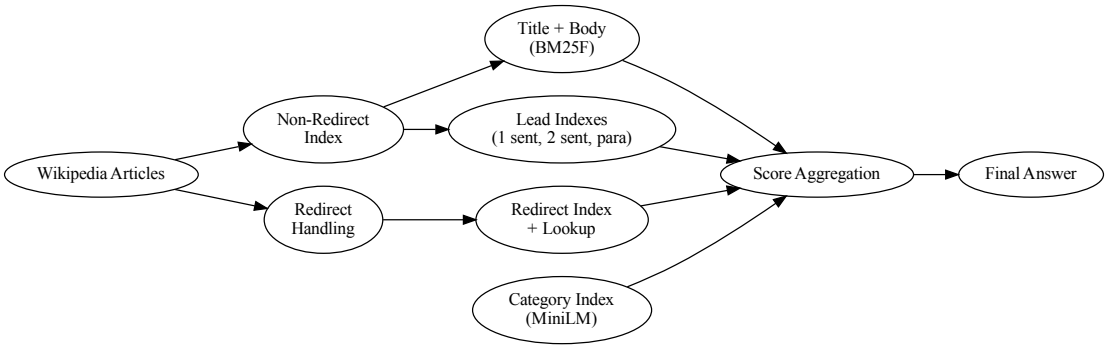

Figure 1: Final three-stage pipeline. A Jeopardy category and clue are first mapped to a weighted hybrid candidate ranking over Wikipedia. The top 100 candidates are reranked by a cross-encoder, the top 10 candidates from the combined second-stage ranking are reranked by Qwen3-Reranker-4B, and the highest-ranked final page title is returned as the answer.

3.4 Stage 2: Cross-Encoder Reranking

The second stage reranks the top 100 candidates from the weighted hybrid retriever using a cross-encoder model, `cross-encoder/ms-marco-MiniLM-L-12-v2` (33M). Each candidate article is represented by its title and cleaned article body, split into 100-token chunks. For each of the five generated natural questions, the system scores all chunks and keeps the best chunk score for that question. The final cross-encoder score for the candidate is the average across the five natural questions. The exact query and document text formats are shown in Appendix Figure 2.

We then combine the cross-encoder score with the initial first-stage score. Both scores are normalized within the top-100 candidate set, and the selected combination uses a weight of 1.00 for the initial score and 7.00 for the cross-encoder score. This stage is still local and relatively efficient because it only reads 100 candidates per clue, but it is much more semantically expressive than stage 1.

3.5 Stage 3: Qwen3-Reranker-4B

The final stage is an offline reranking experiment performed on the top 10 candidates from the combined second-stage ranking. For each clue, we export the top 10 candidates together with the category, clue, gold answer, five generated natural questions, and a truncated evidence snippet from each candidate article. These exported rows are then scored with Qwen3-Reranker-4B.

We also tried a separate Qwen3-4B chat-model verifier that used a direct yes/no prompt over the same evidence snippets. That approach was not effective as a final ranking method, whereas Qwen3-Reranker-4B, which is a dedicated reranker rather than a chat model, performs much better in the final-stage reranking setting (Section 5.1). The exact chat-verifier prompt and reranker input structure are listed in Appendix Figure 2. This final stage cannot recover a correct answer that was not already present in the exported top 10, so it is a pure answer-selection stage rather than a recall-improvement stage.

3.6 Compute Environment

We developed and ran the project across two environments: a local MacBook setup and Google Colab. The local machine was used for the main retrieval codebase, indexing, weight tuning, and most iterative debugging. Google Colab was used for the larger model runs, especially the Qwen-based question generation, summary generation, and reranking experiments.

3.7 Implementation Workflow

We used ChatGPT extensively during implementation to generate code templates, propose project-directory structure, suggest model choices, and help debug basic errors. In particular, it was useful for scaffolding new components quickly, suggesting the cross-encoder reranking approach, surfacing candidate models through search guidance, and helping us think through alternative retrieval designs. We also used it as a consultation tool while deciding how to organize the codebase and which experimental branches were worth implementing.

However, the generated code was not used blindly. We still had to put substantial amount of time reading the generated code carefully, correcting subtle mistakes, and adapting many pieces to the actual behavior of the Wikipedia data and our retrieval pipeline. In practice, the LLM assistance reduced implementation time and improved development speed, but it also required manual review and debugging to make the final code correct and coherent.

4 Evaluation Setup

As mentioned earlier, we evaluate on the same 100 Jeopardy clues used throughout development, over the final indexed collection of 152,042 non-redirect Wikipedia articles.

The final evaluated system uses the Jeopardy category together with the clue text as the initial query. It first produces a global candidate ranking with the weighted hybrid retriever described in Section 3, then reranks the top 100 candidate pages with the cross-encoder by averaging scores across five generated natural questions, and finally reranks the top 10 candidates from that second-stage ranking with Qwen3-Reranker-4B.

We evaluate by normalized title match between the top-ranked predicted page and the gold answer. The primary metric is Precision@1 (Top-1 accuracy), because the system must commit to one final answer. We report Precision@5 as a supporting metric to show whether the correct page is at least being brought near the top of the ranking before the final answer decision.

We directly tuned retrieval weights and selected configurations on the same set of 100 Jeopardy questions used for evaluation. We did not use a separate training or development split, because the dataset is small. As a result, the reported scores should be interpreted as tuned performance on this 100-question set rather than as an unbiased estimate of generalization to unseen clues.

5 Results and Analysis

5.1 Main Results

Table 2 presents the main system-stage comparison. Precision@1 is the primary metric because it directly measures whether the final predicted page title is correct. Precision@5 is included only as supporting context.

The largest early gains come from better retrieval. The sparse-only baselines reach P@1 scores of 0.21 and 0.22, while the full weighted hybrid configuration reaches 0.31. This shows that most of the early improvement comes from stronger candidate generation rather than from later reranking alone.

Table 2: Main system-stage comparison. Precision@1 is the primary metric; Precision@5 is supporting.

System stage	P@1	P@5
Sparse body-only baseline, no category	0.21	0.42
Sparse body-only baseline + category	0.22	0.46
Weighted hybrid retrieval + category	0.31	0.50
Cross-encoder only	0.40	0.62
Combined first-stage + cross-encoder	0.41	0.61
Qwen chat verifier	0.04	0.29
Qwen3-Reranker-4B	**0.47**	**0.58**

Table 3: Top-1 correct counts after grouping the 30 Jeopardy categories into 9 super-categories.

Super-category	N	Vanilla	Weighted	Cross-enc.	Fused	Qwen
People / Biography	19	2	9	10	11	14
Geography / Places	17	0	4	4	5	8
History / Politics	25	0	4	10	8	9
Entertainment / Pop Culture	14	1	6	7	7	5
Business / Companies	5	0	2	0	0	2
Media / Journalism	5	1	2	2	2	3
Organizations / Institutions	3	1	2	2	2	2
Language / Wordplay	3	0	0	0	0	1
Mixed / General Knowledge	9	1	2	5	6	3
Total	**100**	**6**	**31**	**40**	**41**	**47**

The cross-encoder then improves answer selection inside the retrieved set. Using only the cross-encoder ranking over the top 100 candidates reaches a P@1 score of 0.40, and the combined second-stage ranking reaches 0.41. This indicates that many first-stage errors are ranking mistakes rather than complete retrieval misses.

The final Qwen reranker gives the best Top-1 result, improving the P@1 score from 0.41 to 0.47. However, this gain should be interpreted correctly: Qwen3-Reranker-4B only sees the exported top 10 candidates from the combined second-stage ranking. It improves final answer selection inside that top-10 set, but it does not improve recall beyond what earlier stages already retrieved.

One negative result is also important. The Qwen chat verifier performs very poorly, reaching only a P@1 score of 0.04. A yes/no chat probability is therefore not a useful final ranking signal for this task, even when the underlying model is much larger than the cross-encoder.

5.2 Grouped Category Analysis

To make the category-level behavior interpretable, we group the 30 original Jeopardy categories into 9 super-categories. Table 3 shows Top-1 counts at each stage.

The grouped analysis shows that the weighted hybrid retriever is especially important for *People / Biography*, *Entertainment / Pop Culture*, and *Geography / Places*. These categories often depend on identifying a named entity, role, or location from article-body evidence, which is well matched by a strong sparse retriever with dense support.

The cross-encoder helps most on *History / Politics*, where the jump from 4 to 10 correct answers is the largest stage-to-stage improvement in the grouped table. These clues often require better local semantic alignment between a clue-like question and a supporting passage. The cross-encoder is much less reliable on *Business / Companies*, where semantically related company, product, and subsidiary pages are strong distractors.

Qwen reranking helps most on *People / Biography* and *Geography / Places*, where it adds three more correct top-ranked answers in each group. This suggests that once the correct answer is already inside the top 10 and the evidence snippet is explicit, the larger reranker can make better final distinctions than the cross-encoder alone. In contrast, *Entertainment / Pop Culture* and *Mixed / General Knowledge* become worse under Qwen reranking, indicating that the final yes/no-style evidence judgment is less stable for broad, indirect, or comparative clues.

5.3 What Did Not Work

Several explored ideas did not become part of the final system. We tried a custom tokenization and normalization pipeline with explicit lowercasing, stop-word removal, punctuation stripping, and token splitting, but Whoosh's default analyzer performed better in the final sparse index. Lead-only indexes based on the first sentence, first two sentences, or first paragraph all underperformed full-body retrieval as standalone signals. Title-only and redirect-only retrieval also failed as standalone rankers.

We also explored clue paraphrasing for sparse retrieval, but it did not help enough. The paraphrases tended to preserve the same key nouns and proper nouns while only changing surface wording, so they did not create enough new lexical evidence for BM25-style matching. Finally, several other implemented components, including qwen-summary dense indexes and category-semantic variants, remained at zero weight in the selected configuration because they did not improve the final weighted hybrid pipeline enough to justify inclusion.

6 Conclusion

We show that Jeopardy-style Wikipedia answer selection benefits from a staged retrieval design in which stronger models are applied only after the candidate set has already been narrowed. The first-stage weighted hybrid retriever is responsible for the largest early gains because it brings the correct page into a competitive rank range. The cross-encoder then improves semantic ranking within the top 100 candidates, and the final Qwen3-Reranker-4B stage further improves answer selection within the top 10 candidates from the combined second-stage ranking.

The final system reaches a P@1 score of 0.47, compared with 0.31 after initial weighted hybrid retrieval and 0.41 after the combined cross-encoder stage. This supports a clear design lesson: retrieval quality comes first, but larger semantic rerankers become valuable once the system has already produced a strong candidate set at low cost.

The remaining failures follow a small number of patterns. Some questions are still true retrieval misses, where the correct page is not present in the candidate set. Others are ranking misses caused by broad-topic distractors or semantically related wrong pages. Wordplay categories remain difficult because the clue may have little direct lexical overlap with the answer page. Finally, the Qwen reranker is limited by the exported top-10 evidence set, so it cannot recover answers that earlier stages failed to surface.

References

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.

Qwen Team. Qwen3 technical report, 2025.

A Prompt and Text Templates

Figure 2: Prompt and text templates used in the implemented pipeline and related experiments.

Component	Where used	Template or format
Sparse query text	Stage 1 retrieval	Category and clue concatenated into one query string: `{category} {clue}`.
Natural-question generation with Qwen3-8B	Question rewriting before stage 1 and later reranking	System: "You rewrite Jeopardy-style clues into natural search and QA questions. Return valid JSON only." User: "Write exactly 5 natural-language questions from the category and clue ... Use only the information in the category and clue ... Do not reveal the answer ... Return exactly this JSON shape and nothing else: {"questions":["...","...","...","...","..."]}." Input fields: `Category: {category}` and `Clue: {clue}`.
Cross-encoder query text	Stage 2 reranking	Each of the five generated natural questions is used directly as the query text.
Cross-encoder document text	Stage 2 reranking	`Title: {title}\nArticle: {cleaned_article_chunk}`
Qwen summary prompt	Qwen-summary dense-index experiment	System: "You summarize Wikipedia-style articles for retrieval. Preserve unique facts, names, dates, places, aliases, and distinguishing details. Return valid JSON only." User: "Write one concise factual summary for dense retrieval. Use 3 to 5 sentences ... Return exactly this JSON shape and nothing else: {"summary":"..."}." Input fields: `Title: {title}` and `Article text: {article_text}`.
Chat-verifier prompt	Failed Qwen3-4B chat-model verifier	`Question: {natural_question}\nArticle: {evidence}\n\n==========\n\nDoes this article contain the answer to the question? Answer with exactly one word: Yes or No.`
Reranker content text	Final Qwen3-Reranker-4B stage	`<Instruct>: {instruction}\n<Query>: {natural_question}\n<Document>: {evidence}`
Reranker instruction	Final Qwen3-Reranker-4B stage	"Given a Jeopardy clue rewritten as a natural question, retrieve articles that contain enough information to identify the answer." The model is further wrapped with a system prefix that restricts the decision to "yes" or "no."